When the Index Shapes the Agent: Exact Memory Retrieval for Generative Agents with HNSW

An extension of the seminar presentation on Malkov and Yashunin (2020)

Colceriu Mihai Bogdan

Master in Software Engineering, Technical University of Cluj-Napoca

Intelligent Agent Systems (Sisteme de agenți inteligenți), 2026–2027

Abstract

Generative agents (Park et al., 2023) decide what to recall by scoring every memory with a weighted sum of recency, importance and relevance. In practice the memories sit in a vector index such as HNSW (Malkov and Yashunin, 2020), the article presented at the seminar. A vector index answers a single question: which stored vectors are most similar to the query. Agent frameworks therefore fetch the K most similar memories and re-rank them by the full score, so a memory that is recent or important but not similar enough is never considered. This report first reimplements HNSW from the paper's pseudocode, checks it against the authors' library and repeats the paper's parameter study: the hierarchy pays off in low dimensions, while on clustered and real data the neighbour-selection heuristic matters more. It then shows that, because recency decays exponentially, the agents' score is exactly an inner product between augmented vectors: a single inner-product HNSW index can search directly for the top-k memories under the full score, with weights and time chosen per query. A memory store built on this idea keeps the extra coordinates bounded with time-anchored segments, supports refresh-on-access through a write buffer and handles the paper's min-max normalisation by rescaling the weights. With one million memories it keeps recall@10 at 0.975 (against 0.006 for the re-ranking used by LangChain) while answering a query roughly a hundred times faster than a linear scan. In a closed-loop simulation of 25 agents calibrated to the paper's party scenario, the folded store reproduces the behaviour of exact retrieval. In towns older than a week, plain similarity search cuts the spread of the invitation by about 40%, while LangChain-style re-ranking changes it in a direction that depends on how large the memories have grown. Finally, running the public Generative Agents code shows that its recency term is computed from ranks sorted in the wrong direction, so the memory accessed longest ago receives the highest recency score.

1 Introduction

Agents built on language models need a memory that outlives the context window. The usual design keeps a stream of natural-language records, embeds each one as a vector and retrieves the records that matter for the current situation (Lewis et al., 2020; Wang et al., 2024). Generative agents (Park et al., 2023) made the retrieval rule explicit: a memory is worth recalling if it was accessed recently, if it is important and if it is relevant to the query. The three terms are added after min-max scaling.

The index underneath is very often a graph built with HNSW (Malkov and Yashunin, 2020). It finds the stored vectors most similar to a query in roughly logarithmic time, but it knows nothing about recency or importance. Frameworks work around this by fetching more candidates than they need and re-ranking them. LangChain's `TimeWeightedVectorStoreRetriever` (LangChain, 2026), for example, fetches 100 documents by similarity, adds the most recently created ones and returns the best k under the combined score. A memory outside this candidate set is never scored, however recent or important it is. As the memory grows, the candidates cover a shrinking fraction of it. Nothing tells the agent that it is now retrieving something different from what its retrieval rule describes.

This report starts from the HNSW article and asks whether the index itself can answer the agent's real question. It can. The reduction is exact, so the only approximation left is the index's own. The contributions

are the following.

1. A reimplementation of HNSW from Algorithms 1–5 of the paper, validated against hnswlib and used to repeat the paper's parameter study on synthetic data and on GloVe vectors (section 3).
2. An exact reduction of the Generative Agents score to maximum inner product search (MIPS), obtained by combining forward decay (Cormode et al., 2009) with an inner-product HNSW index, together with a memory store that makes it practical: bounded coordinates, refresh-on-access, min-max normalisation and a replacement for LangChain's retriever with the same scoring rule (section 4).
3. An evaluation on up to one million memories (section 5) and in Willowbrook, a closed-loop simulation of 25 agents calibrated to reproduce the information diffusion reported by Park et al. (2023) (section 6).
4. A check of the public Generative Agents code, which computes recency differently from the paper (section 7).

The individual ingredients are known. The factorisation of exponential decay into a query part and an item part is the basis of forward decay in stream processing. Graph indexes for inner products were studied by Morozov and Babenko (2018). What this report adds is their combination for agent memory, the engineering needed for memories whose scores change every time they are retrieved and evidence that the choice of retrieval method changes what a population of agents does.

2 Background

2.1 Approximate nearest neighbour search and HNSW

Given n vectors in $\mathbb{R}^d$ and a query, k-nearest neighbour search returns the k stored vectors closest to the query. For the dimensions of text embeddings an exact answer costs time linear in n, so practical systems accept approximate answers and measure their quality as recall@k, the fraction of the true k nearest neighbours that is returned. Approximate methods partition the space (trees, or inverted files with vector quantisation as in FAISS (Johnson et al., 2021)), hash it, or build a proximity graph and search it greedily. Graph methods have led the ANN-Benchmarks comparisons for years (Aumüller et al., 2020).

Navigable small world (NSW) graphs (Malkov et al., 2014) insert elements one by one and connect each new element to its nearest neighbours among those already inserted. Early elements acquire long links that later act as shortcuts, so greedy routing reaches the target in few hops. The number of hops still grows polylogarithmically with n. The high-degree early nodes also slow the search down on low-dimensional or clustered data.

HNSW separates the links by length into layers, as a skip list does with sorted keys (Pugh, 1990). Every element draws a top layer $l = \lfloor -\ln(U)\, m_L \rfloor$ with U uniform on $(0, 1]$, so the population of each layer shrinks exponentially. A search starts at the single entry point on the top layer, descends greedily (beam width 1) to layer 1 and then runs a best-first search with a beam of ef candidates on layer 0 (Algorithms 2 and 5 of the paper). Insertion is the same search followed by linking the new element to M neighbours on each of its layers (Algorithm 1). The neighbours are not simply the M closest: the heuristic of Algorithm 4 accepts a candidate only if it is closer to the new element than to every neighbour accepted so far, which keeps the links pointing in different directions and preserves the connections between clusters. An element keeps at most $M_{\max} = M$ links on the upper layers and $M_{\max 0} = 2M$ on layer 0. The paper recommends $m_L = 1/\ln M$. Under idealised assumptions the number of distance evaluations per query grows as $O(\log n)$.

2.2 Memory retrieval in generative agents

A generative agent stores what it perceives, says and concludes as timestamped natural-language records in its memory stream (Park et al., 2023). When it plans its day, talks to another agent, reacts to an event or reflects, it retrieves memories for a focal point q by scoring every memory i as

$$\text{score}_i \;=\; a\,\widehat{\text{rec}}_i + b\,\widehat{\text{imp}}_i + c\,\widehat{\text{rel}}_i, \tag{1}$$

	recall@10		dist. eval.	queries / s	
ef	ours	hnswlib	ours	ours	hnswlib
16	0.5817	0.5703	618	5 189	17 688
32	0.6891	0.6869	961	3 268	10 493
64	0.7780	0.7773	1572	2 102	6 516
128	0.8436	0.8456	2705	1 046	3 736
256	0.8958	0.8968	4824	571	1 365
512	0.9403	0.9420	8756	315	906
build time				669 s	235 s

Table 1: The reimplementation against hnswlib on GloVe-100 (same $M = 16$, $ef_{\text{construction}} = 200$, 1 000 queries, one thread).

where recency $\text{rec}_i = \delta^{T-t_i}$ decays with the game hours since the memory was last accessed ($\delta = 0.995$), importance imp_i is a rating from 1 to 10 that the language model gives the memory when it is created (scaled to $[0,1]$ here) and relevance $\text{rel}_i = \langle q, e_i \rangle$ is the cosine similarity between the unit embeddings of the focal point and of the memory. The hats denote min-max scaling over all memories. The paper sets $a = b = c = 1$. A retrieved memory has its access time reset to the current time, so the score of a memory depends on the agent's own history of retrievals.

2.3 Retrieval in practice

Vector databases only answer similarity queries, so the combined score is usually computed on a candidate set. LangChain's `TimeWeightedVectorStoreRetriever` (LangChain, 2026) scores a document as $(1-\lambda)^h + \sum(\text{metadata scores such as importance}) + \text{relevance}$, where h is the number of hours since the last access, but only for the $K = 100$ documents returned by a similarity search and for the k most recently created documents, which enter without a relevance term. It does not normalise the terms. In the rest of the report this pattern is called *overfetch and re-rank*.

3 HNSW from the paper

3.1 Implementation

Algorithms 1–5 are implemented in Python and compiled with numba, following the pseudocode closely (`agentmem/paper_hnsw.py`, about 250 lines). Levels are drawn in advance, layer-0 links live in an $n \times 2M$ array and upper-layer links in a second array addressed through per-element offsets. Every distance evaluation is counted, which gives a cost measure that does not depend on the machine or the compiler. The parameter study below uses it instead of wall-clock time. One detail differs from hnswlib: during insertion the paper passes the whole result list W of one layer on as entry points to the next, whereas hnswlib keeps only the closest element. The reimplementation follows the paper.

3.2 Validation

Table 1 compares the reimplementation with hnswlib on 500 000 GloVe vectors (100 dimensions, cosine similarity, the vectors behind ANN-Benchmarks' `glove-100-angular`) with $M = 16$ and $ef_{\text{construction}} = 200$. Recall@10 agrees within 0.016 at every search depth. hnswlib answers about 3.2 times as many queries per second, as expected from a C++ library with SIMD distance kernels compared with a numba prototype.

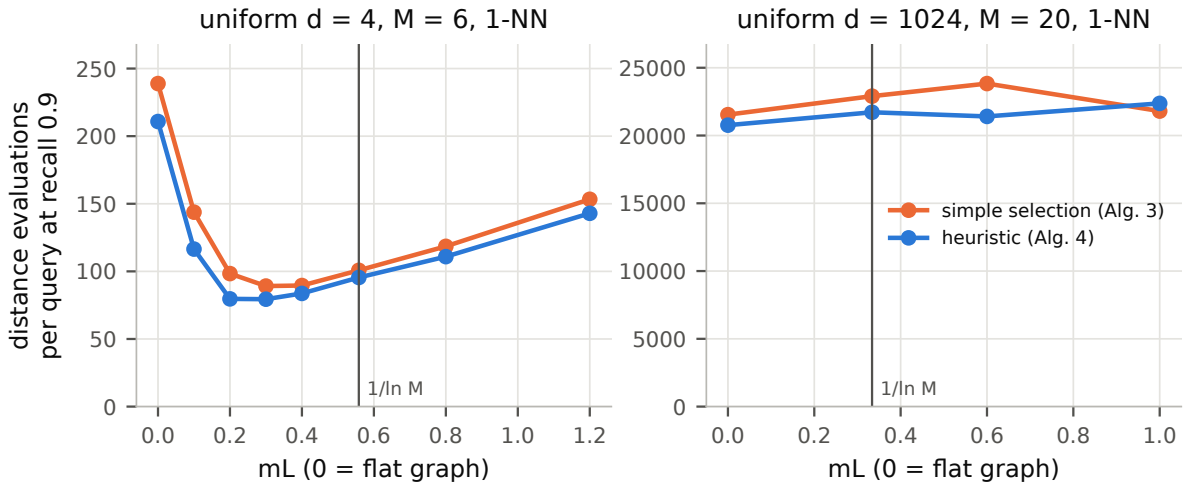


Figure 1: Cost of reaching recall 0.9 (1-NN) as a function of m_L, for the simple and the heuristic neighbour selection. The vertical line marks the paper's default $m_L = 1/\ln M$.

3.3 The paper's parameter study, repeated

Figures 3–7 of the paper study the hierarchy and the neighbour heuristic on random and real data. Repeated at a smaller scale (one million points instead of ten million, 50 000 points instead of 100 000 at $d = 1024$, GloVe instead of SIFT), they give the same picture (figs. 1 and 2).

- **Low dimension.** On one million uniform points with $d = 4$ the hierarchy pays off. A flat graph ($m_L = 0$) needs 211 distance evaluations per query to reach recall 0.9, the best setting ($m_L \approx 0.3$) needs 79 and the paper's default $1/\ln M \approx 0.56$ needs 95: close to the optimum, though not at it. The gain grows with the number of points: the flat graph needs 2.7 times as many evaluations as the best hierarchy here, against 1.7 times in a trial run with 50 000 points (`results/smoke`). This is in line with the paper's argument that the hierarchy turns polylogarithmic search into logarithmic search.
- **High dimension.** At $d = 1024$ every setting needs between 20764 and 23834 evaluations, almost half of the 50 000 points, because distances between uniform random points in so many dimensions are nearly all the same. The hierarchy and the heuristic make no difference beyond noise. The paper reports a small gain from the hierarchy in this setting, which we do not reproduce at half its size.
- **Clustered data.** With 100 isolated clusters in $d = 10$, the simple selection (the M closest candidates) never gets past recall 0.765: clusters end up linked only internally and the greedy search cannot leave the cluster where it starts. The heuristic keeps a few links between clusters and reaches 0.996. On uniform data the two are close.
- **Real data.** On GloVe a flat graph is as good as the hierarchical one (5066 against 5101 evaluations at recall 0.9). This agrees with Munyampirwa et al. (2024), who argue that in high dimensions a few hub nodes form a highway that does the job of the upper layers. The heuristic still matters: the simple selection needs 27% to 34% more evaluations.

4 Folding the agent's score into the index

4.1 An exact inner-product form

Leave the normalisation aside for a moment and consider the unnormalised score $a\,\mathrm{rec}_i + b\,\mathrm{imp}_i + c\,\mathrm{rel}_i$. Because the decay is exponential it factorises around any anchor time τ,

$$\delta^{T-t_i} = \delta^{T-\tau}\,\delta^{\tau-t_i}, \tag{2}$$

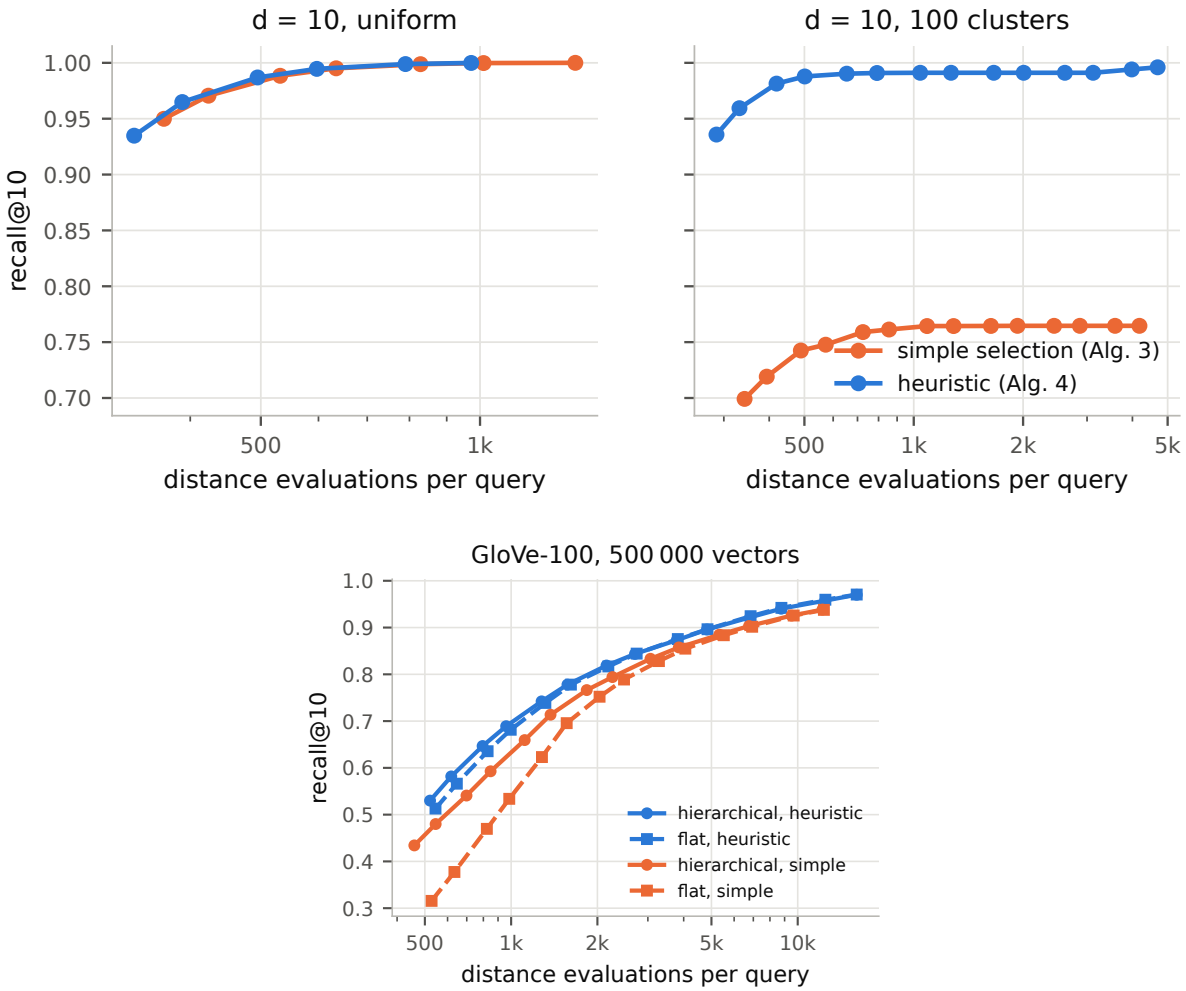


Figure 2: Top: recall against cost on one million points in $d = 10$, uniform and with 100 isolated clusters. Bottom: flat against hierarchical graphs on GloVe-100.

where the first factor depends only on the query time and the second only on the memory. This is the idea behind forward decay in stream processing (Cormode et al., 2009). For a construction scale $s > 0$ define

$$x_i = \begin{bmatrix} e_i \\ s\,\delta^{\,\tau - t_i} \\ s\,\mathrm{imp}_i \end{bmatrix}, \qquad Q = \begin{bmatrix} c\,q \\ (a/s)\,\delta^{\,T-\tau} \\ b/s \end{bmatrix}, \qquad \langle Q, x_i \rangle \;=\; a\,\delta^{\,T-t_i} + b\,\mathrm{imp}_i + c\,\langle q, e_i \rangle. \tag{3}$$

The identity holds for every memory, every query time and every choice of weights. Retrieving the best k memories is therefore a maximum inner product search over $(d+2)$-dimensional vectors. HNSW can be built with the inner product as its similarity (Morozov and Babenko, 2018). The memory vectors depend neither on the weights nor on the query time, so the same index serves an agent whose weights or clock change from one query to the next.

4.2 Min-max normalisation

Min-max scaling subtracts the minimum of each component and divides by its range. For a given query the minima are constants that do not change the ranking. The ranges only rescale the weights:

$$\mathrm{score}_i = \frac{a}{R_{\mathrm{rec}}}\,\mathrm{rec}_i + \frac{b}{R_{\mathrm{imp}}}\,\mathrm{imp}_i + \frac{c}{R_{\mathrm{rel}}}\,\mathrm{rel}_i - \mathrm{const}. \tag{4}$$

The normalised top-k is therefore the unnormalised top-k with weights $(a/R_{\mathrm{rec}}, b/R_{\mathrm{imp}}, c/R_{\mathrm{rel}})$. Two of the ranges are cheap to maintain: R_{imp} from the smallest and largest importance and $R_{\mathrm{rec}} = \delta^{\,T-t_{\max}} - \delta^{\,T-t_{\min}}$

from the latest and earliest access times (a heap with lazy deletion tracks $t_{\min}$ while access times change). The relevance range depends on the query. Its maximum is the best match of q and its minimum is the best match of $-q$, so two extra searches with $k = 1$ on the same index estimate it.

4.3 From a formula to a memory store

Three practical problems remain.

Bounded coordinates. A memory accessed after the anchor has $\delta^{\tau - t_i} > 1$, so the coordinate grows exponentially: by a factor of about 50 000 over 90 days with $\delta = 0.995$. Anchoring at the latest access time keeps every coordinate in $(0, 1]$, but new memories keep arriving. The store therefore writes into a *hot segment* anchored at its creation time and closes it after one week, so its coordinates stay below $\delta^{-168} \approx 2.3$. When more than four closed segments exist, they are merged into a single index re-anchored at its latest access time, which also drops deleted entries. A query searches every segment, so it matters that their number stays bounded.

Refresh on access. Each retrieval resets the access time of the memories it returns, which changes their recency coordinate. The store marks them as deleted in their segment and moves them into a write buffer of 256 memories that is scored by a linear scan. A full buffer is flushed into the hot segment.

Exact re-scoring. The candidates from all segments and from the buffer are merged and scored with the exact formula. With LangChain's unnormalised score the approximation can therefore only miss memories, never reorder the ones it found. Under min-max scaling the candidates are ranked with the weights obtained from the estimated ranges, so they can also come out in a slightly different order from the exact score. Section 5 shows that this is where most of the remaining loss comes from.

The inner product in eq. (3) is exact for any s, but the graph is built on the vectors x_i, so s decides how much the two metadata coordinates shape the neighbourhoods. With a small s the graph is essentially the semantic graph. With a large s it groups memories by age and importance. Section 5 measures the effect. The default is $s = 1$.

4.4 A retriever for LangChain

`ExactTimeWeightedRetriever` scores documents like LangChain's retriever and keeps its main fields (decay rate, additional score keys such as importance, k, stamping of the access time), but keeps the documents in the folded store. It is not a complete drop-in replacement: it takes an embedding model instead of a vector store, has no asynchronous interface and reads the additional score keys when a document is added. LangChain does not normalise, so no range estimates are needed and the reduction is exact.

5 Retrieval benchmark

5.1 Setup

Data. The scale test uses GloVe vectors (Twitter, 100 dimensions, unit length) as memory embeddings with synthetic agent metadata: creation times uniform over 90 days, 5% of the memories accessed again later, importance drawn from the distribution observed in the town simulation of section 6. Metadata and embeddings are independent here, which is the least favourable case for similarity-based candidates. The second data set has realistic correlations: the pooled memory streams of the 25 Willowbrook agents after 92 simulated days (126 454 memories with simulated importance and access times) queried with the focal points the agents actually use.

Methods. *Exact scan* scores every memory (NumPy, the ground truth). *Relevance only* returns the k most similar memories from an HNSW index, which is what a plain retrieval-augmented memory does. *Overfetch and re-rank* fetches K memories by similarity, adds the k newest and re-ranks them with the full

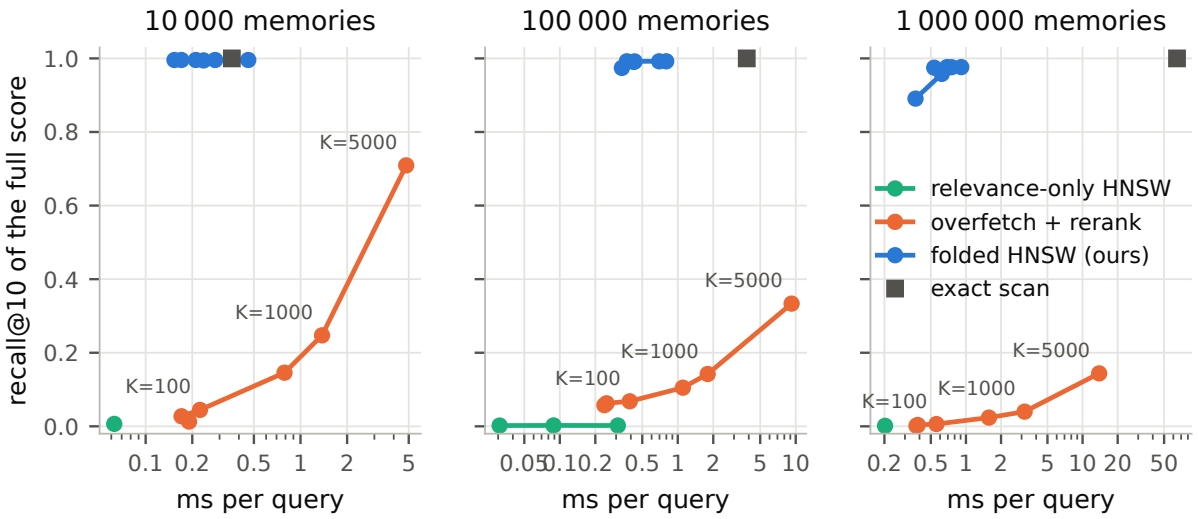


Figure 3: Recall@10 of the paper's score against query time on GloVe with synthetic metadata. Points along a curve are search depths (ef) or, for re-ranking, candidate counts K.

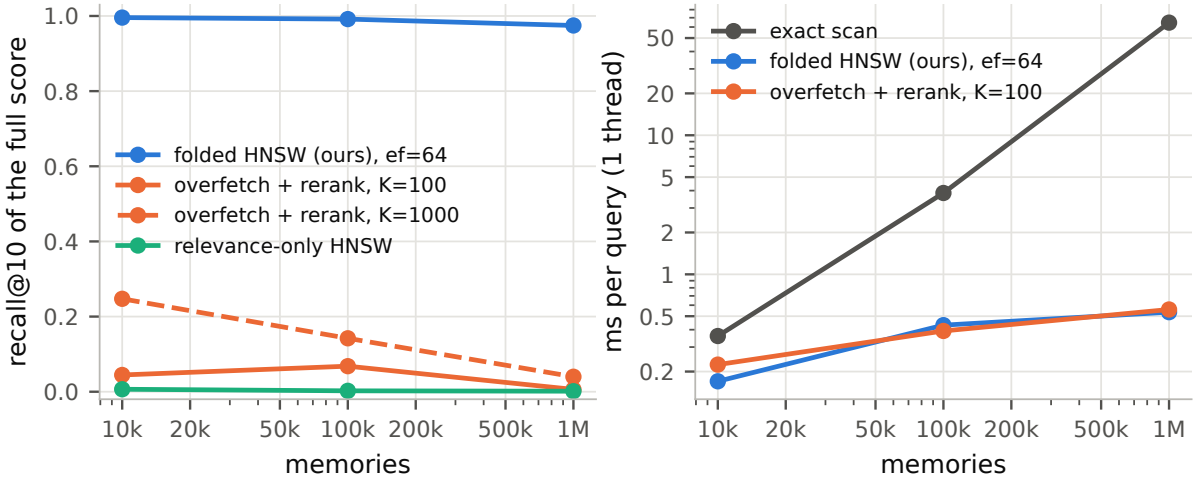


Figure 4: Recall (left) and query time (right) as the memory grows.

score, with the normalisation ranges estimated in the same way as for our store. *Folded HNSW* is the store of section 4. All HNSW indexes use hnswlib with $M = 16$ and $ef_{\text{construction}} = 200$. Queries run on one thread and recall@10 is measured against the exact top-10 of the same score. The machine is a cloud virtual machine with two cores.

5.2 Results

Figures 3 and 4 and table 2 summarise the benchmark. Three observations stand out.

Similarity candidates miss most of the true top-k. With the paper's weights, the 10 memories with the highest score are mostly recent and important ones, rarely among the most similar. Relevance-only retrieval recovers 0.001 of them with one million memories and overfetch with $K = 100$ recovers 0.006. More candidates help, but slowly: $K = 1000$ reaches 0.040. With $K = 100$ it stays low at every size: 0.045, 0.068 and 0.006 at 10 000, 100 000 and one million memories. On the town memories, where similarity and recency are correlated, the gap is smaller but still large (0.277 for $K = 100$).

The folded index recovers almost all of the true top-k. At $ef = 64$ the folded store reaches 0.975 on

	GloVe, 1 000 000 memories				Willowbrook, 126 454 memories		
Method	paper	code	raw	ms	paper	code	raw
Exact scan	1.000	1.000	1.000	64.5	1.000	1.000	1.000
Relevance only	0.001	0.016	0.002	0.20	0.042	0.045	0.043
Overfetch + re-rank, $K = 100$	0.006	0.083	0.010	0.56	0.277	0.325	0.290
Overfetch + re-rank, $K = 1000$	0.040	0.298	0.059	3.18	0.661	0.769	0.729
Folded HNSW, $ef = 64$	0.975	0.875	0.996	0.53	0.959	0.978	0.994
Folded HNSW, $ef = 256$	0.976	0.959	1.000	0.69	0.962	0.981	0.999

Table 2: Recall@10 and query time with one million GloVe memories and on the pooled Willowbrook memories, for the three weight profiles: the paper's (min-max, $a = b = c = 1$), the public code's weights (min-max, recency 0.5, importance 2, relevance 3) and LangChain's unnormalised sum. Folded HNSW and relevance only use $ef = 64$.

	GloVe, 1 000 000		Willowbrook, 126 454	
range search	recall@10	ms	recall@10	ms
beam 32 (default)	0.971	0.47	0.967	0.16
beam 64	0.982	0.57	0.970	0.17
beam 128	0.988	1.16	0.982	0.40
beam 256	0.991	1.70	0.984	0.38
exact ranges ($O(n)$ scan)	0.997	–	0.990	–

Table 3: Recall@10 of the folded store ($ef = 64$, paper's weights) and query time as the beam of the two searches that estimate the relevance range grows. The last row uses the exact range. The index is a separate build from the one in table 2.

one million memories and 0.959 on the town memories. A deeper graph search hardly changes this (0.976 at $ef = 256$), because most of the remaining loss comes from the estimated relevance range. With the exact ranges, an $O(n)$ scan used here only for the analysis, the same index reaches 0.997 and 0.990. The two range searches use a beam of 32 by default. Table 3, from a separate build of the same indexes, shows that a wider beam closes most of the gap, at the price of a slower query: with a beam of 128, recall on one million memories rises from 0.971 to 0.988 and the query time from 0.47 to 1.16 ms.

With the weights of the public code, which put most of the weight on relevance (0.5 for recency, 2 for importance and 3 for relevance), the folded index needs a deeper search on GloVe: 0.875 at $ef = 64$ and 0.959 at $ef = 256$. Section 5.3 explains why.

Index construction runs on two threads and is not deterministic, so two builds of the same index differ slightly. On the town memories, 7 builds (`experiments/exp_buildvar.py` plus the builds behind tables 2 and 3) gave recall between 0.959 and 0.968 at $ef = 64$ with the paper's weights and between 0.972 and 0.983 with the code's weights. Differences of this size between the tables are noise.

It stays fast. A linear scan of one million memories takes 64.5 ms per query on one core when NumPy evaluates the full formula and about 46 ms for the matrix-vector product alone, the least an exact scan can cost. The folded store answers in 0.53 ms, 87 to 121 times faster. Its query time grows slowly with the size of the memory (fig. 4).

Ablations (fig. 5). Anchoring at the oldest access time instead of the newest makes the recency coordinate span almost five orders of magnitude, so recall collapses to 0.058 whatever the search depth. The construction scale matters in both directions: $s = 0.25$ hides the metadata from the graph (0.259 at $ef = 128$), $s = 4$ lets it dominate (0.863) and $s = 1$ gives 0.992. The classic reduction of MIPS to Euclidean search (Bachrach et al., 2014) needs a deeper search than the inner-product graph for the same recall, which matches Morozov and Babenko (2018). On the town memories it ends higher at large depths, though (0.981 against 0.963 at $ef = 512$), probably because its two range searches work better on a metric graph. The fold itself is not tied to HNSW. An IVF-Flat index from FAISS on the same vectors works too, with a worse trade-off on GloVe (0.907 at 0.89 ms with 64 probes, against 0.992 at 0.37 ms for HNSW) and a better one on the town

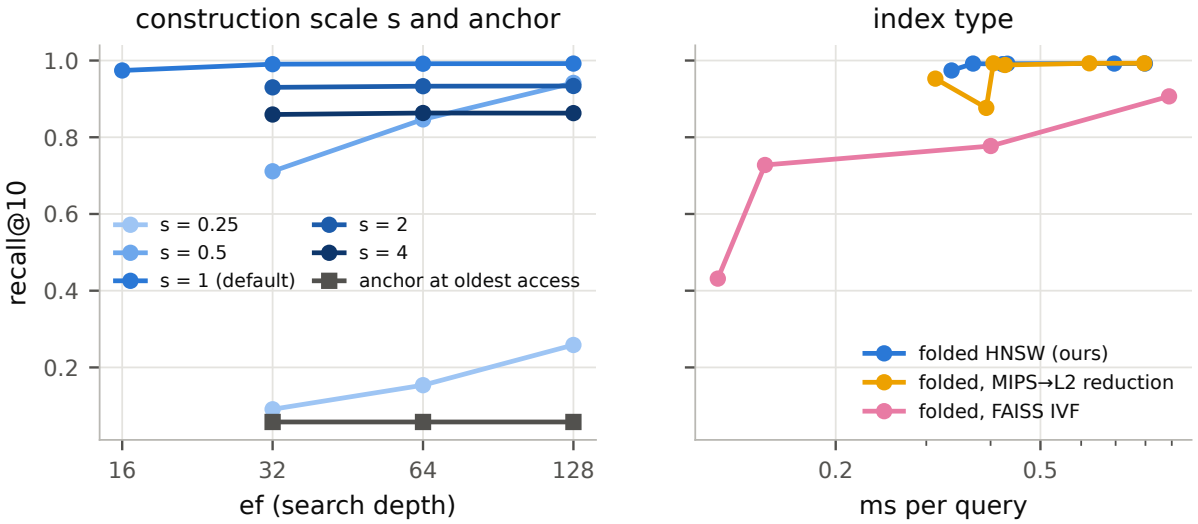


Figure 5: Ablations on 100 000 GloVe memories with the paper's weights. Left: construction scale s and the choice of anchor. Right: the same folded vectors in an inner-product HNSW index, in an HNSW index after the MIPS-to-Euclidean reduction and in a FAISS IVF-Flat index.

		overfetch + re-rank		folded HNSW (s, ef)			
relevance weight	rel. only	K=100	K=1000	s=1, 64	s=1, 256	s=0.5, 64	s=0.5, 256
0.25	0.001	0.096	0.123	0.996	0.996	0.768	0.985
0.5	0.001	0.081	0.118	0.994	0.994	0.816	0.985
1	0.002	0.068	0.142	0.992	0.993	0.849	0.983
2	0.018	0.093	0.245	0.983	0.988	0.848	0.978
4	0.105	0.336	0.660	0.909	0.969	0.854	0.970
8	0.366	0.699	0.954	0.759	0.909	0.835	0.956
16	0.634	0.848	0.983	0.695	0.868	0.814	0.938
32	0.746	0.858	0.970	0.706	0.854	0.805	0.924

Table 4: Recall@10 on 100 000 GloVe memories as the relevance weight grows (recency and importance weights fixed at 1, min-max normalised).

memories with the paper's weights (0.996 at 0.31 ms with 16 probes), though not with the code's weights (0.800, against 0.978 for HNSW). With exact ranges HNSW reaches a similar 0.990 on the paper's weights, which suggests that the advantage of IVF there comes from its range searches.

5.3 When are similarity candidates enough?

Overfetching is not always wrong. Table 4 raises the weight of relevance while keeping recency and importance at 1. Up to a relevance weight of 2 the folded index stays at or above 0.983 at $ef = 64$, while overfetch never exceeds 0.245, even with $K = 1000$. Between 4 and 8 the picture turns. Once relevance outweighs the other two terms several times over, the top-k is mostly made of the most similar memories: at weight 8 overfetch with $K = 1000$ reaches 0.954 and the folded index 0.759 ($ef = 64$) or 0.909 ($ef = 256$). At weight 16 even $K = 100$ (0.848) does better than the folded index at $ef = 64$ (0.695). The graph of the folded index was shaped partly by the metadata coordinates, so a query that ignores them needs a deeper search or a smaller construction scale. A store that serves both kinds of queries can route them by their weights, or use $s = 0.5$, which stays above 0.924 at $ef = 256$ across the whole sweep.

5.4 LangChain

The comparison with LangChain uses the real classes: the memory streams of five Willowbrook agents after 30 days (1 308–1 920 documents each), importance as an additional score key, cosine similarity as relevance, a decay rate of 0.005 per hour so that recency matches the paper's 0.995 (LangChain's default is 0.01) and the agents' own planning and conversation focal points as queries. Measured against the exact top-10 of LangChain's own formula, LangChain's retriever returns 0.518 of the right documents and our retriever returns 1.000.

6 Agents in a town

6.1 Willowbrook

Willowbrook has twelve places (a café, a library, a college, a park, a pub, a town hall and others) and 25 residents with jobs, homes, weekly routines, favourite places and interests. Time advances in steps of 15 minutes. The cognitive loop follows Generative Agents, with every call to the language model replaced by a rule.

- **Perception.** At every step a resident observes what it is doing and what up to three other people at the same place are doing. New observations enter its memory stream with an importance that depends on the activity (sleeping 1, routine at home 2, work or leisure 3, chatting 4, attending an event 7) plus a small random offset.
- **Conversation.** Residents at the same place talk with a probability that depends on their sociability. Each of the two in turn retrieves memories with the focal points of the reference code's `converse.py`: the other's name ($k = 50$) and "*other* is *activity*" ($k = 25$, as in `agent_chat_v1`). Both versions of the reference code also use a relationship summary written by the language model as a focal point, which Willowbrook leaves out. The version the code actually calls, `agent_chat_v2`, uses $k = 15$ for the second retrieval and repeats both retrievals before every utterance. With probability 0.5 the speaker then passes on the most important piece of news among the retrieved memories that it has not yet exchanged with the other, otherwise the two make small talk. Both remember the conversation.
- **Planning.** At wake-up every resident retrieves memories with the focal points of `plan.py`, "*name*'s plan for *date*." and "Important recent events for *name*'s life." ($k = 30$ each). At 14:00 it retrieves again with a focal point of our own, "*name*'s plan for this evening, *date*." ($k = 30$). If an event of that day is among the retrieved memories, the resident attends it with a fixed probability (0.45 for the party, 0.35 for other events).
- **News.** Every eight hours somebody learns a piece of gossip (a new puppy, a promotion, a picnic on Thursday). The scenario of the paper is reproduced on top of this: Maria Lopez, who runs the café, learns 35 hours in advance that she is hosting a party there at 5 pm. She invites everyone she talks to.

Every retrieval refreshes the access time of the memories it returns. Random decisions use counter-based hashing, so two runs with the same seed see the same world and diverge only where their retrievals differ.

Calibration. Three parameters have no counterpart in the paper. The probability of attending the party once it comes up when planning was set to 0.45 to match the 5 of 12 invited agents who came in the paper and was kept fixed. The other two, the conversation rate and the probability of passing on news, were chosen with exact retrieval in a young town (two days of memories) so that the outcome matches the Valentine's Day party of Park et al. (2023), where 13 of the 25 agents knew about the party after two game days and 5 of the 12 invited agents came. Over a grid of 15 settings with 12 seeds each (`experiments/calibrate_town.py`), a conversation rate of 0.06 and a sharing probability of 0.5 give 13.4 knowing agents and 5.1 guests on average.

Experiment. The independent variable is how long the town has lived before the scenario. A run lasts 2 days (as in the paper), 9, 32 or 92 days. The party always takes place at 5 pm on its last day. Maria learns about it 35 hours earlier, when the town is 0, 7, 30 or 90 days old, so in longer runs every agent carries

a larger memory into the scenario. Runs are named by their length below. Each length was run with 30 seeds and four retrieval methods, all with the paper's score (min-max, $a = b = c = 1$): the exact scan, the folded store, overfetch with $K = 100$ and relevance only. The outcomes are the number of agents who knew about the party before it started, who had it among their retrieved memories when planning the day and who attended.

6.2 Results

Figure 6 and table 5 show the outcomes. In a young town the four methods agree: with about a hundred memories per agent, the party ranks high under any of them. From nine days on they drift apart, because the approximate methods return a shrinking share of the paper's top-k as the memories grow. Averaged over the 92-day runs they return 0.487 (overfetch) and 0.366 (relevance only) of it. By the end of the runs this falls to about 0.32 and 0.26 (fig. 7, right).

The folded store behaves like the exact scan. Over the 92-day runs its retrievals contain 0.999 of the paper's top-k on average. The three outcomes are identical on every seed up to 32 days and on 26 of the 30 seeds at 92 days. On the other seeds they differ by at most 2 agents, in both directions.

Relevance-only memory loses about 40% of the diffusion. From nine days on, 38% to 43% fewer agents know about the party and 33% to 45% fewer attend it (8.2 against 13.1 knowing agents and 3.3 against 5.8 guests at 92 days). The invitation travels through conversations. The focal point of a conversation is the listener's name, which the party is rarely similar to. Without recency and importance the agents do not think of mentioning it.

Overfetch and re-rank moves the outcome in both directions. In the 9-day runs it spreads the news slightly less than the exact scan (11.2 against 11.7 knowing agents, Holm-adjusted $p = 0.037$). In the 32-day runs the gap looks similar (10.9 against 11.7) but is within noise after the correction ($p = 0.37$). In the 92-day runs it spreads the news clearly more: 15.2 agents know about the party and 13.3 recall it when planning, against 13.1 and 10.8, although it returns less than half of the paper's top-k. The reason is the second half of its candidate set, the newest memories. A piece of news is among an agent's newest memories right after it is heard, so it always becomes a candidate, while most of the recent observations that would outrank it under the full score are neither that new nor similar enough to the focal point and never reach the re-ranking. Taking the newest memories out of the candidate set confirms this. With only the 100 most similar memories, the outcome at 92 days falls to 8.3 knowing agents and 3.8 guests (7.0 and 3.3 at 32 days), close to relevance-only memory, although the share of the paper's top-k it retrieves hardly changes (0.502 against 0.487).

Two conclusions follow. An approximate retrieval does not lose information at random: it changes which information the agents act on. The direction of the change depends on details such as which recent memories enter the candidate set and how large the memory has grown. And recall against the paper's top-k does not predict the effect, since two methods with the same recall led the town to very different outcomes.

7 The reference implementation

In the public code of Generative Agents (Park, 2023), retrieval is the function `new_retrieve` of `retrieve.py`. Running that function unmodified on mock personas, with only the OpenAI embedding call replaced, shows three differences from Section 4.1 of the paper.

1. Recency depends on rank, not on time: the memories are sorted by access time and the j-th one receives δ^j, whatever the number of hours between them ($\delta = 0.995$ for the 25 agents of the paper's main scenario, 0.99 by default).
2. The sort is ascending, so the memory accessed longest ago comes first and receives the highest recency. The memory accessed most recently receives the lowest. After min-max scaling these become 1 and 0 (fig. 7).
3. The weights are 0.5 for recency, 3 for relevance and 2 for importance instead of all ones.

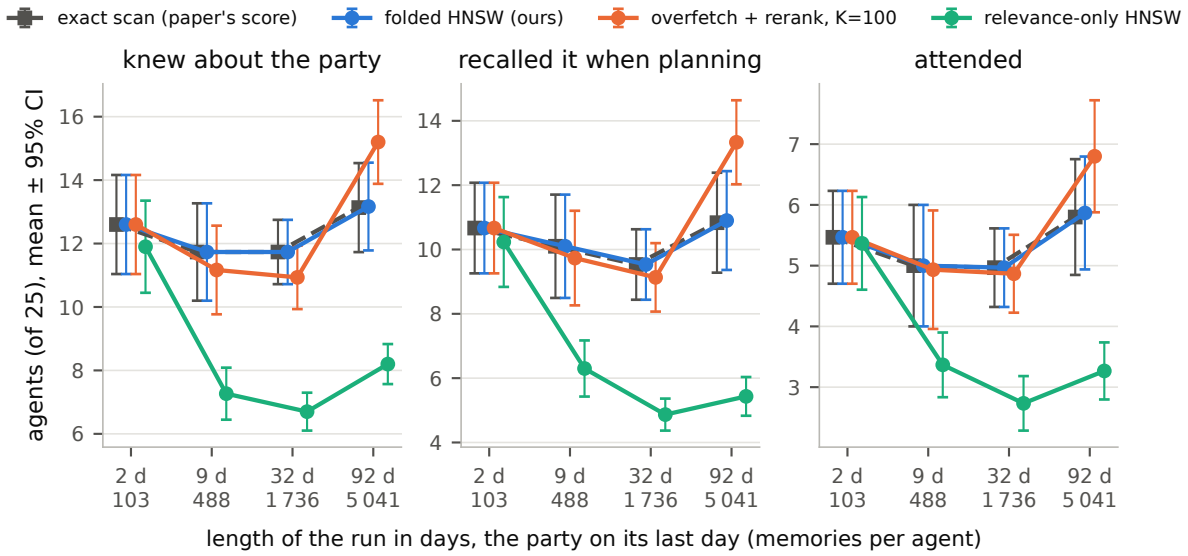


Figure 6: The party scenario in runs of increasing length (30 seeds each, mean and 95% confidence interval). The folded store reproduces the exact scan on every seed up to 32 days and on 26 of 30 at 92 days.

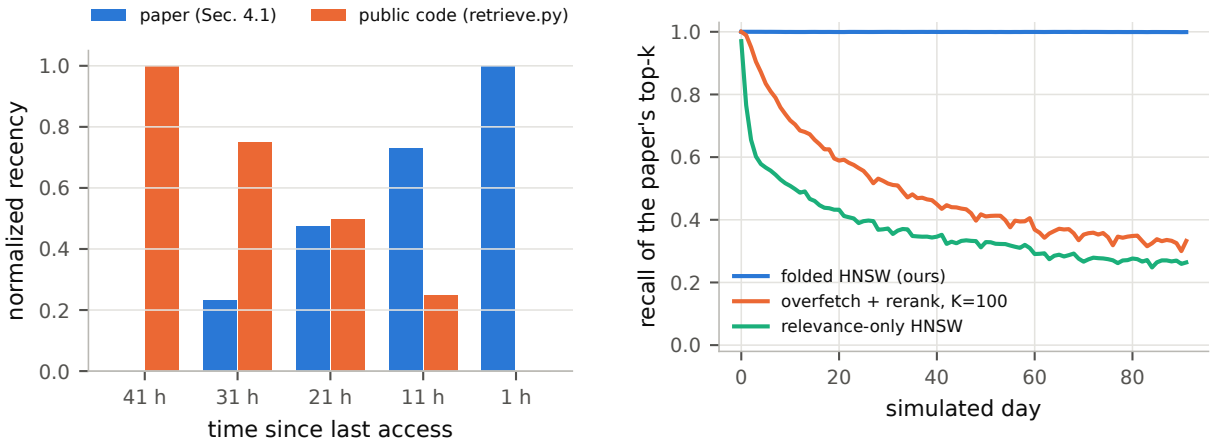


Figure 7: Left: normalised recency of five memories that differ only in their last access time, according to the paper and according to the public code. Right: recall of the paper's top-k at every retrieval in the 92-day Willowbrook runs.

The heavier relevance weight dampens the effect of the reversed recency. In the town, for the planning focal points of 25 agents at the end of a 32-day run, the reference code shares 89% of its top-30 with the paper's formula, but the memories it retrieves were last accessed 17.6 hours earlier on average, against 8.6 hours for the paper's formula. Part of that gap comes from the weights. With the code's own weights and the paper's recency the average is 10.0 hours and the overlap with the reference code 95%, so the reversed recency alone raises it from 10.0 to 17.6 hours. The comparison says nothing about which version produced the results in the paper. It does mean that anyone reusing the public code gets a different recency term from the one the paper describes.

8 Limitations

- The agents of Willowbrook follow rules where Generative Agents call a language model. This keeps the runs deterministic, free and repeatable: the town experiments in this report took 720 runs. The comparison only needs the agents to act on what they retrieve. It also means that the size of the behavioural effects is specific to these rules. An optional script (`experiments/llm_validation.py`) repeats the party

Run	Retrieval	memories	recall	knew	recalled	attended
2 days	Exact scan (paper)	103	1.000	12.6 ± 1.6	10.7 ± 1.4	5.5 ± 0.8
	Folded HNSW (ours)	103	1.000	12.6 ± 1.6	10.7 ± 1.4	5.5 ± 0.8
	Overfetch + re-rank, K=100	103	0.995	12.6 ± 1.6	10.7 ± 1.4	5.5 ± 0.8
	Relevance only	103	0.872	$11.9 \pm 1.5^{**}$	$10.2 \pm 1.4^{*}$	5.4 ± 0.8
9 days	Exact scan (paper)	488	1.000	11.7 ± 1.5	10.1 ± 1.6	5.0 ± 1.0
	Folded HNSW (ours)	488	1.000	11.7 ± 1.5	10.1 ± 1.6	5.0 ± 1.0
	Overfetch + re-rank, K=100	487	0.876	$11.2 \pm 1.4^{*}$	9.7 ± 1.5	4.9 ± 1.0
	Relevance only	484	0.637	$7.3 \pm 0.8^{**}$	$6.3 \pm 0.9^{**}$	$3.4 \pm 0.5^{**}$
32 days	Exact scan (paper)	1736	1.000	11.7 ± 1.0	9.5 ± 1.1	5.0 ± 0.6
	Folded HNSW (ours)	1736	1.000	11.7 ± 1.0	9.5 ± 1.1	5.0 ± 0.6
	Overfetch + re-rank, K=100	1734	0.681	10.9 ± 1.0	9.1 ± 1.1	4.9 ± 0.6
	Relevance only	1725	0.488	$6.7 \pm 0.6^{**}$	$4.9 \pm 0.5^{**}$	$2.7 \pm 0.4^{**}$
92 days	Exact scan (paper)	5041	1.000	13.1 ± 1.4	10.8 ± 1.6	5.8 ± 1.0
	Folded HNSW (ours)	5041	0.999	13.2 ± 1.4	10.9 ± 1.5	5.9 ± 0.9
	Overfetch + re-rank, K=100	5034	0.487	$15.2 \pm 1.3^{*}$	$13.3 \pm 1.3^{**}$	6.8 ± 0.9
	Relevance only	5007	0.366	$8.2 \pm 0.6^{**}$	$5.4 \pm 0.6^{**}$	$3.3 \pm 0.5^{**}$

Table 5: The party scenario by length of the run. Memories per agent at the end of the run, mean recall of the paper's top-k over all retrievals and the number of agents (of 25) who knew about the party, recalled it when planning and attended, as mean $\pm$ 95% confidence interval (t distribution) over 30 seeds. Stars mark a difference from the exact scan in a paired Wilcoxon test, Holm-corrected over the 36 comparisons in the table ($^{*}p < 0.05$, $^{**}p < 0.01$).

scenario with planning and conversation decisions taken by a local language model.

- Sentence embeddings come from GloVe with smooth inverse frequency weighting (Arora et al., 2017) because the machine used here cannot download neural encoders. The store is independent of the encoder.
- The scale test attaches synthetic metadata to GloVe vectors. All timings come from a two-core virtual machine and vary by up to about 30% between runs.
- For a single agent's memory in the town (5 041 memories after 92 days) an exact scan is still faster than the folded store written in Python (1.77 against 1.90 ms per retrieval). In the static benchmark the folded store is already faster at 10 000 memories (fig. 4).
- The folded index loses recall when relevance dominates the score, where plain similarity search is adequate anyway (table 4).

9 Conclusion

HNSW answers similarity queries. Agents ask a different question. Because recency decays exponentially, the question asked by generative agents is an inner product in disguise. A single inner-product HNSW index can answer it directly, with the weights and the time chosen per query. Making this work for a living memory takes three ingredients: time anchors with segments, a buffer for memories that were just retrieved and a cheap estimate of the normalisation ranges. The resulting store returns almost all of the paper's top-k where the usual overfetch-and-re-rank pattern returns only a small part of it. In a simulated town that difference changes how far news travels and who turns up at a party. The reimplementation of HNSW reproduces the paper's parameter study except for a small gain from the hierarchy at $d = 1024$. It also agrees on 100-dimensional GloVe vectors with the more recent observation that the hierarchy matters less than the neighbour heuristic.

A Reproducing the results

All code, data loaders, tests and experiment scripts are in the accompanying repository, `https://github.com/bogdancolceriu/agentmem-hnsw` (see `README.md`). The GloVe vectors are downloaded

from the gensim-data release on first use. The 26 unit tests check that the folded inner product equals the score up to rounding, that with a deep search the store returns more than 99% of the memories of the exact scan over long sequences of queries with refresh, segment compaction and min-max normalisation, that segment coordinates stay bounded and that the LangChain retriever returns at least 7 of the 8 documents of a brute-force search for every query.

B Use of AI tools

This work was done with substantial help from an AI assistant (Claude, Anthropic), which wrote most of the code, ran the experiments and drafted this report. I chose the article, discussed the design of the extension with the assistant, checked the code and the results and can reproduce every experiment from the repository. Every result in the text and in the tables is generated from the result files by `experiments/tables.py`, so none was copied by hand.

References

Sanjeev Arora, Yingyu Liang, and Tengyu Ma. A simple but tough-to-beat baseline for sentence embeddings. In *International Conference on Learning Representations (ICLR)*, 2017.

Martin Aumüller, Erik Bernhardsson, and Alexander Faithfull. ANN-Benchmarks: A benchmarking tool for approximate nearest neighbor algorithms. *Information Systems*, 87, 2020.

Yoram Bachrach, Yehuda Finkelstein, Ran Gilad-Bachrach, Liran Katzir, Noam Koenigstein, Nir Nice, and Ulrich Paquet. Speeding up the xbox recommender system using a euclidean transformation for inner-product spaces. In *Proceedings of the 8th ACM Conference on Recommender Systems (RecSys)*, pages 257–264, 2014.

Graham Cormode, Vladislav Shkapenyuk, Divesh Srivastava, and Bojian Xu. Forward decay: A practical time decay model for streaming systems. In *Proceedings of the 25th IEEE International Conference on Data Engineering (ICDE)*, pages 138–149, 2009.

Jeff Johnson, Matthijs Douze, and Hervé Jégou. Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, 7(3):535–547, 2021. arXiv:1702.08734.

LangChain. TimeWeightedVectorStoreRetriever, langchain-classic. https://github.com/langchain-ai/langchain, 2026. file libs/langchain/langchain_classic/retrievers/time_weighted_retriever.py, accessed October 2026.

Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

Yu. A. Malkov and D. A. Yashunin. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 42(4):824–836, 2020. arXiv:1603.09320.

Yury Malkov, Alexander Ponomarenko, Andrey Logvinov, and Vladimir Krylov. Approximate nearest neighbor algorithm based on navigable small world graphs. *Information Systems*, 45:61–68, 2014.

Stanislav Morozov and Artem Babenko. Non-metric similarity graphs for maximum inner product search. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2018.

Blaise Munyampirwa, Vihan Lakshman, and Benjamin Coleman. Down with the hierarchy: The 'H' in HNSW stands for "hubs", 2024. arXiv:2412.01940.

Joon Sung Park. generative_agents: reference implementation of "generative agents: Interactive simulacra of human behavior". https://github.com/joonspk-research/generative_agents, 2023. commit fe05a71, Apache-2.0.

Joon Sung Park, Joseph C. O'Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology (UIST)*, 2023.

William Pugh. Skip lists: a probabilistic alternative to balanced trees. *Communications of the ACM*, 33(6): 668–676, 1990.

Lei Wang, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, Jiakai Tang, Xu Chen, Yankai Lin, Wayne Xin Zhao, Zhewei Wei, and Jirong Wen. A survey on large language model based autonomous agents. *Frontiers of Computer Science*, 18(6), 2024.